

Project 계획서

프로젝트명	국문	등교시간 예측 서비스 '나가자'		
	영문	Nagaja – School Departure Time Prediction Service		
학부(과)	응용소프트웨어공학과		담당교수	김삼문
프로젝트 수행팀	팀명	가드		
	학번	성명	연락처(휴대폰)	E-mail
	20212999	허재경	010-6333-6273	jeje9893@gmail.com
	20213021	곽우석	010-9312-0583	ognam400040@gmail.com
	20200372	김종호	010-4487-3289	ho098701@naver.com
프로젝트수행기간		2026. 3. 2 - 2026. 6. 30(4개월)		
동의대학교 응용소프트웨어공학과 관계규정과 지시사항을 준수하면서 Novice Project 프로젝트를 성실히 수행하고자 하며, 이에 아래와 같이 프로젝트계획서를 제출합니다. 2026년 3월 29일 프로젝트팀장 : 허재경 (서명) 담당(지도)교수 : 김삼문 (서명) 동의대학교 소프트웨어공학부(과)장 귀하				

〈 요약 문 〉

프로젝트 목표 (300자내외)	사용자의 등교 패턴, 실시간 교통 상황, 날씨 등의 다양한 데이터를 종합적으로 분석하여 개인에게 최적화된 출발 시간(준비 시간)을 예측하는 서비스를 개발하는 것을 목표로 한다. 이를 통해 지각 가능성을 사전에 줄이고 이동 시간을 효율적으로 관리할 수 있도록 지원하며, 직관적인 인터페이스와 맞춤형 알림 기능을 제공하여 사용자 편의성과 실생활 활용도를 극대화하고자 한다.		
내용 (500자내외)	본 프로젝트는 사용자 등교 데이터를 기반으로 실시간 교통 API, 날씨 데이터, 대중교통 혼잡도 정보를 수집·통합하여 최적의 출발 시간을 예측하는 서비스를 개발하는 것을 목표로 한다. 데이터 수집 및 전처리 과정을 거쳐 머신러닝 기반 예측 알고리즘을 구현하고, 사용자별 준비 시간, 이동 패턴, 지각 이력 등을 반영한 개인 맞춤형 예측 모델을 구축한다. 기존의 단순 경로 및 소요 시간 안내 서비스와 달리, 실제 사용자 행동 데이터를 기반으로 개인별 생활 패턴을 학습하여 맞춤형 결과를 제공하는 것이 핵심 차별점이다. 또한 모바일 애플리케이션과 IoT 기반 물리적 알람시계를 연동하여 알람 해제 시각과 출발 시각(버튼 입력 및 Wi-Fi 연결 상태 감지)을 수집하고 서버와 실시간 동기화함으로써 데이터의 신뢰도를 높인다. 특히 물리적 디바이스를 활용하여 사용자의 실제 행동 데이터를 자동으로 수집·반영하는 점에서 기존 소프트웨어 중심 서비스와의 차별성을 갖는다. 이를 통해 사용자에게 최적화된 준비 시작 시간과 출발 전략을 제시하고, 시간 관리 효율성과 서비스 정확도를 동시에 향상시키는 것을 지향한다.		
기대효과 (200자내외)	본 서비스를 통해 사용자는 개인 맞춤형 출발 시간 안내를 받아 지각 가능성을 효과적으로 줄이고, 효율적인 시간 관리가 가능해진다. 또한 실제 행동 데이터를 반영한 예측 정확도 향상을 통해 서비스 신뢰도를 높이고, IoT 디바이스 연동으로 사용자 편의성과 몰입도를 동시에 향상시킬 수 있다.		
Keywords	지각	알람	예측 서비스
	개인별 맞춤	시간 관리	IoT

목 차

I. 서론.....	1
1. 프로젝트의 배경 또는 필요성.....	1
2. 프로젝트의 목표.....	1
II. 프로젝트 수행 내용.....	2
1. 프로젝트의 내용	2
2. 프로젝트 추진전략 및 방법(체계).....	9
III. 결론.....	10
1. 프로젝트 결과의 활용방안.....	10
2. 프로젝트 결과의 기대효과.....	10
[참고문헌].....	11

I. 서론

1. 프로젝트의 배경 또는 필요성

1) 프로젝트에 대한 현재 사회적 동향 및 기술 상황

최근 바쁜 현대인들에게 시간관리는 삶의 질을 결정하는 중요한 요소입니다. 기술의 발전으로 실시간 교통 정보와 날씨 데이터가 API 형태로 개방되면서, 이를 활용한 도착 예정 시간 안내 서비스가 보편화되었습니다. 그러나 기존 서비스들은 단순히 현재 시점에서의 소요 시간만을 제공할 뿐, 사용자가 실제로 집에서 나가기 위해 준비하는 시간이나 개인별 이동 습관을 충분히 반영하지 못하고 있습니다.

2) 기존 문제점 분석 및 새로운 요구사항

- 개인별 생활패턴 미반영: 기존 지도 서비스는 도로 상황만을 고려하며, 사용자가 세면, 옷 입기 등 등교를 위해 소요하는 '준비 시간'은 계산에서 제외됩니다.
- 실제 행동 데이터의 부재: 소프트웨어 기반 서비스는 사용자가 정확히 언제 출발했는지에 대한 실제 데이터를 수집하기 어려워 예측 정확도에 한계가 있습니다.
- 유동적 알림 부족: 갑작스러운 폭우나 사고로 인한 교통 체증 발생 시, 사용자가 평소보다 일찍 일어날 수 있도록 유동적으로 대응하는 맞춤형 서비스에 대한 요구가 높아지고 있습니다.

3) 프로젝트의 필요성

이러한 한계를 극복하기 위해 사용자의 실제 등교 패턴과 외부 환경(교통, 날씨 등)을 결합한 지능형 예측 서비스가 필요합니다. 특히 모바일 앱과 IoT 기반의 알람시계를 연동함으로써 사용자의 실제 기상 및 출발 데이터를 수집하고, 이를 머신러닝으로 학습하여 개인 맞춤형 출발시간을 제시하는 서비스가 필요합니다.

2. 프로젝트의 목표

- 사용자의 행동 데이터와 실시간 환경 데이터를 종합 분석하여 지각 가능성을 최소화하고, 개인별 맞춤형 준비 시작 및 출발시간을 안내하는 IoT 연동 지능형 등교 지원 서비스 '나가자' 개발.
- (1) 공학 : '기존 도착 예정시간' 대비 '실제 도착시간' 오차율 15% 이내 개선.
IoT 기기(Wi-Fi 연결 감지 등)를 활용한 출발 시각 자동 기록 성공률 95% 이상 확보.
외부 환경 변화(폭우, 사고 등) 감지 시 사용자에게 가변적 알람을 제공.

II. 프로젝트 수행내용

1. 프로젝트의 내용

1) 목표 달성을 위한 프로젝트의 구체적인 내용

(1) 공학 : 설계 과정에서 고려해야 하는 구성요소 및 현실적인 제한조건

본 서비스 '나가자'는 사용자의 시간표, 위치, 교통수단, 개인 준비시간을 기반으로 최적 출발 시각을 실시간으로 계산하고 행동을 유도하는 개인 맞춤형 등교 출발 시간 + 행동 유도 시스템이다. 단순한 알람 앱이 아닌, 전용 물리 알람시계와 모바일 앱이 연동되어 사용자의 실제 행동 데이터를 누적·학습하는 습관 교정 서비스에 가깝다.

서비스는 크게 5개의 모듈로 구성된다.

① 사용자 데이터 모듈

사용자별 맞춤 예측의 기반이 되는 데이터를 저장·관리한다. 요일별 시간표, 출발지·목적지 위치, 선택 교통수단, 개인 준비시간(초기값 → 누적 학습값)을 포함하며, DB 구조는 아래와 같이 설계한다.

```
User → Schedule (요일별 수업시간)
      → Location (home, school 좌표)
      → Transport (버스 / 지하철 / 도보)
      → PrepTime (초기값 + 실측 history)
```

② 시간 계산 엔진 (서비스의 핵심 두뇌)

출발 시각 및 준비 시작 시각을 도출하는 핵심 로직으로, 아래 공식을 기반으로 동작한다.

```
출발 시각 = 수업 시간 - (평균 이동시간 + 날씨 보정값)
준비 알람 시각 = 출발 시각 - 개인 준비시간
```

이동시간은 길찾기 API(Kakao Maps / Google Maps)를 통해 실시간으로 산출하며, 강수·적설 시 +10~15분의 날씨 보정값을 적용한다. 교통수단별 가중치도 반영하여 버스는 배차 대기시간을 추가 산정하고, 지하철은 정시성이 높으므로 보정 없이 적용하며, 도보는 날씨 영향을 가장 크게 반영한다. 초기에는 rule-based로 구현하고, 이후 ML 기반 개인화 예측으로 확장할 수 있도록 구조를 설계한다.

③ 물리 알람시계 + 앱 연동 모듈 (하드웨어)

본 프로젝트의 핵심 차별점으로, 라즈베리파이(Raspberry Pi) 기반의 전용 물리 알람시계 디바이스를 직접 제작하여 모바일 앱과 Wi-Fi / Bluetooth로 연동한다. 라즈베리파이는 완전한 Linux 환경을 갖추고 있어 Firebase와 직접 통신하고, 날씨·교통 API를 자체적으로 호출하는 등 단순 수신 장치가 아닌 독립적인 엣지 컴퓨팅 디바이스로 동작한다.

물리 시계의 주요 구성은 다음과 같다.

구성요소	역할
Raspberry Pi (4 또는 Zero 2W)	메인 연산, Wi-Fi 통신, Firebase 연동, 알람 로직 실행
디지털 디스플레이 (7-Segment / OLED)	현재 시각 및 출발까지 남은 시간 표시
RGB LED 링 (NeoPixel / WS2812B)	여유(초록) / 나가자(노랑) / 지각(빨강) 상태 색상 표시
물리 버튼 (준비시작 / 출발)	알람 해제 → 준비 타이머 시작 / 출발 시각 기록
부자·스피커	알람 소리 출력 (단계별 볼륨 상승)
GPIO 진동모터	알람 진동 출력

라즈베리파이는 앱 서버(Firebase Firestore)와 Wi-Fi로 상시 연결되어 앱에서 설정한 수업 시간·개인 준비시간 데이터를 실시간으로 수신한다. 계산된 준비 알람 시각이 되면 라즈베리파이가 직접 알람을 실행하고, 사용자가 물리 버튼으로 알람을 끄면 해당 이벤트가 Firestore에 기록되어 앱의 준비 타이머와 동기화된다. 또한 라즈베리파이가 직접 날씨 API를 호출하여 우천 시 LED 색상을 우산 아이콘 등으로 추가 표시하는 독립 기능도 수행한다.

앱과 물리 시계는 Firestore를 중간 허브로 사용하여 데이터를 공유하므로, 인터넷 연결만 있으면 별도의 페어링 없이 항상 동기화된 상태를 유지한다.

④ 실시간 상태 시스템 (UX 핵심)

사용자에게 정확한 숫자보다 "지금 상태"를 직관적으로 전달하는 것이 이 서비스의 핵심 UX 원칙이다. 상태는 아래 3단계로 구분되며, 앱 화면과 물리 시계의 LED가 동일한 색상으로 동기화된다.

상태	조건	앱 화면	물리 시계 LED
 여유	남은 시간 > 준비 + 이동 + 10분	초록 게이지	초록 점등
 나가자	남은 시간 ≈ 준비 + 이동	노란 게이지	노랑 점멸
 지각 위기	남은 시간 < 이동시간	빨간 게이지	빨강 빠른 점멸 + 부저

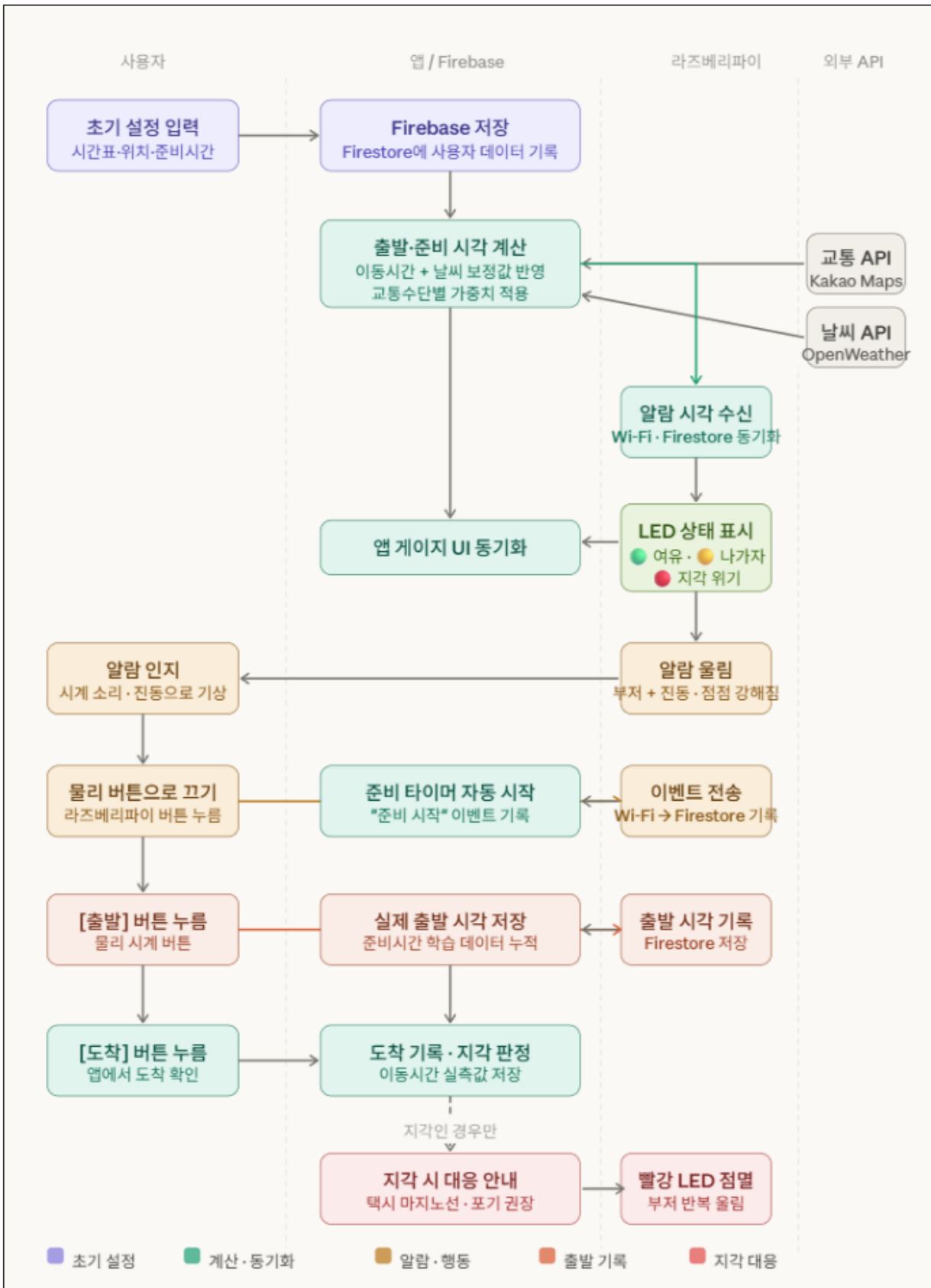
지각 상태 진입 후에는 택시로 이동했을 때 수업에 맞출 수 있는 마지막 시각(택시 마지노선)을 자동 계산하여 앱과 시계 디스플레이에 동시 표시하고, 이 시각마저 지나면 "오늘은 포기" 메시지를 출력한다.

⑤ 알람 & 행동 트래킹 모듈

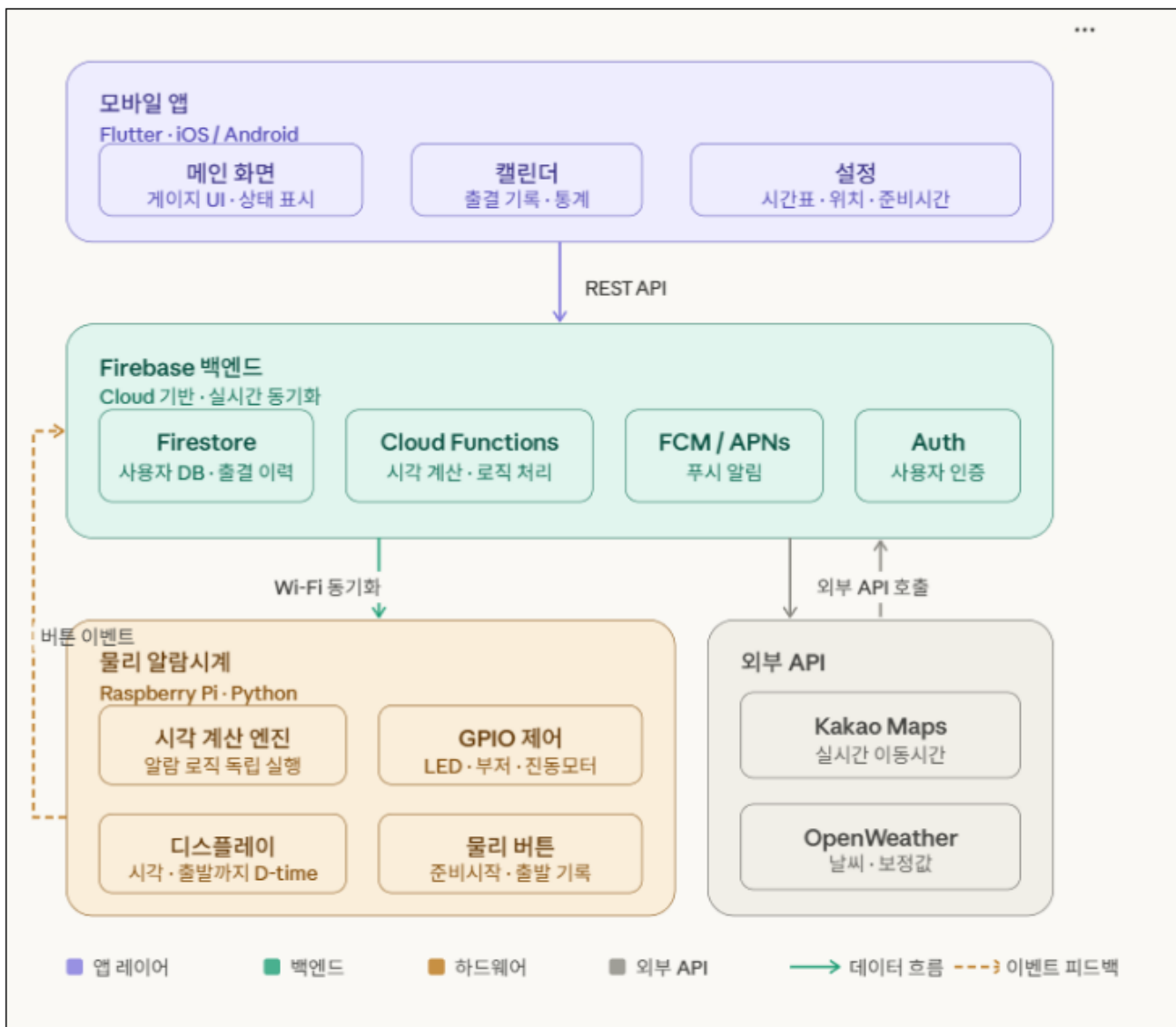
물리 시계의 버튼을 누르는 행동 자체가 "준비 시작"으로 인식되어 타이머가 자동 시작되는 **행동 연동 방식**으로 구성한다. 알람을 끄지 못할 경우 부저 소리와 진동이 점점 강해지며 반복된다. [출발] 버튼을 누르면 실제 출발 시각이 앱에 기록되고, 학교

도착 후 앱의 [도착] 버튼을 누르면 실측 이동시간이 저장되어 이후 예측 정확도 향상에 활용된다. 지각한 날의 원인(준비 지연 / 이동 지연 / 날씨 영향)도 분류·저장하여 추후 추천 정확도를 높이는 데 사용한다.

-서비스 흐름도 (사용자 · 제작자 · 시스템)



-아키텍처 설계도



-화면 구성

화면	주요 구성 요소
메인 화면	실시간 상태 게이지(여유/나가자/지각), 현재 시각, 수업까지 남은 시간, 준비완료·출발 버튼
지각 대응 화면	택시 마지노선 시각 표시, 포기 권장 메시지
캘린더	주간·월간 출결 기록(여유·지각·결석 도트 시각화)
설정	시간표, 출발지·목적지, 교통수단, 개인 준비시간, 시계 BLE 연결 관리

-세부 사용 기술 요소

구분	기술
프론트엔드	Flutter (iOS·Android 크로스플랫폼)
백엔드	Firebase (Firestore, Cloud Functions, FCM, Auth)
지도·이동시간	Kakao Developers API 또는 Google Maps Platform
날씨	OpenWeatherAPI (강수 여부·기온 실시간 반영)
알림	FCM / APNs 푸시 알림 + 물리 시계 부저 연동
하드웨어	Raspberry Pi 4 또는 Zero 2W, RGB LED(NeoPixel WS2812B), OLED 디스플레이, 물리 버튼, 부저, GPIO 진동모터
통신	Wi-Fi — Firestore 실시간 동기화 (앱 ↔ 라즈베리파이)
시계 소프트웨어	Python (RPi.GPIO, rpi_ws281x, Firebase Admin SDK)
데이터 학습	초기 2주 실측값 누적 → 준비시간 자동 보정 (rule-based → ML 확장)

(2) 비공학 : 프로젝트 진행 과정에서 고려해야 하는 내용

- **사용자 조사**: 부산 지역 대학생 대상 설문을 통해 평균 준비시간, 지각 빈도, 지각 원인(준비 지연·교통 지연·날씨)을 수집하여 초기 파라미터를 설정함
- **UX 설계 원칙**: 사람은 정확한 숫자보다 현재 상태(여유/위기)에 반응하여 행동하므로, 숫자 중심이 아닌 상태 중심의 인터페이스를 앱과 물리 시계 모두에 동일하게 적용함
- **하드웨어 비용**: ESP32 모듈, 디스플레이, LED 등 부품 원가를 최소화하여 20,000원 내외로 물리 시계 제작을 목표로 하며, 소프트웨어는 무료 API 및 Firebase 무료 티어를 활용함
- **지속가능성**: 초기 무료 운영 후 광고 없는 구독 모델(월 990원~1,000원) 도입을 검토하며, 물리 시계는 1회 구매 형태로 제공함
- **윤리 및 프라이버시**: 위치 정보는 이동시간 계산 목적에만 사용하며 외부에 제공하지 않음을 명시함

2) 프로젝트 설계의 독창성

- 기존 지도·알람 앱과 달리 **개인 준비시간까지 역산**하여 "지금 준비를 시작해야 할 시각"을 알려주는 차별화된 접근임
- **전용 물리 알람시계를 직접 제작하여 앱과 BLE로 연동**함으로써, 스마트폰을 보지 않아도 LED 색상만으로 현재 상태를 즉각 인지하고 행동으로 이어지도록 유도하는 물리·디지털 통합 시스템을 구현함

- 시계 버튼을 누르는 물리적 행동이 자동으로 앱의 "준비 시작" 이벤트와 연동되는 **행동 기반 UX**로, 사용자가 의식하지 않아도 자연스럽게 행동이 유도됨
- 날씨 데이터와 교통수단별 가중치를 실시간으로 반영하여 상황에 따른 정확한 출발 시각을 제시함
- 지각 시 **택시 마지노선**을 자동 계산하여 현실적인 대안을 즉시 안내함
- 출발·도착 실측 데이터 누적으로 서비스 사용 기간이 길어질수록 예측 정확도가 향상되는 **자기 개선형 구조**를 가짐

3) 기타

- MVP는 시간표 입력 → 위치 입력 → 알람 계산의 3단계로 2~3주 내 앱 구현을 목표로 하며, 이후 물리 시계 하드웨어 제작 및 BLE 연동 → 날씨 반영 → 준비시간 학습 → 통계·리워드 기능 순으로 단계적으로 확장함
- 향후 귀가 시각 예측(약속 후 집까지 나가야 하는 시간 안내) 기능을 추가하여 등교 외 일상 전반의 시간 관리 서비스로 확장할 예정임

2. 프로젝트 추진전략 및 방법(체계)

1) 프로젝트 추진 일정 및 방법

- (1) 본 프로젝트는 총 15주의 일정으로 진행되며, 요구사항 분석 → 시스템 설계 → 핵심 기능 개발(MVP) → 하드웨어 제작 및 연동 → 통합 테스트 및 고도화 → 최종 발표 및 문서화의 6단계로 구성된다.
- (2) 정보 수집 단계(1~3주)에서는 부산 지역 대학생 20명 이상을 대상으로 설문조사를 실시하여 평균 준비시간, 지각 빈도, 지각 원인 등 기초 데이터를 수집하고, 기존 지도·알람 앱과의 비교 분석 및 문헌 조사를 병행한다. 시스템 설계 단계(3~5주)에서는 Flutter 앱 화면 설계, Firebase Firestore DB 스키마 설계, 라즈베리파이 회로도 및 GPIO 핀 맵 설계를 수행한다. MVP 개발 단계(5~9주)에서는 시간 계산 엔진, 앱 메인 화면·알람 기능, Firebase 연동, 외부 API(Kakao Maps, OpenWeather) 연동을 구현한다. 하드웨어 제작 및 연동 단계(7~11주)에서는 라즈베리파이에 OLED 디스플레이, RGB LED, 물리 버튼 등을 조립하고 Python으로 GPIO 제어 코드를 작성하여 앱과 Wi-Fi 실시간 동기화를 구현한다. 통합 테스트 단계(10~13주)에서는 실사용자 베타 테스트를 통해 도착 시간 예측 오차 ± 3 분 이내를 목표로 정확도를 검증하고, 날씨·버스 혼잡도 보정값을 튜닝한다. 최종 발표 단계(13~15주)에서는 보고서 작성, 시연 영상 및 발표 자료를 제작하고 결과물을 GitHub에 업로드한다.

[표 1] 프로젝트 추진 일정

No	프로젝트 활동 내용	추진 일정															기간 (주)	추진 방법
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		
1	요구사항 분석 및 자료 조사																	사용자 설문 조사 (부산 지역 대학생 20명 이상 대상), 선행 서비스(지도알람 앱) 비교 분석, 인터넷 문헌 조사 및 관련 기술 자료 수집
2	시스템 설계 (앱 + 하드웨어)																	Flutter 앱 화면 설계 및 와이어프레임 제작, Firebase DB 구조 설계 (Firestore 스키마),라즈베리파이 회로 도 및 GPIO 핀 맵 설계
3	핵심 기능 개발 (MVP)																	시간 계산 엔진 구현 (출발·준비 시각 공식), Flutter 앱 메인 화면·알람 기능 구현, Firebase Auth-Firebase 연동, OpenWeather-Kakao Maps API 연동 라즈베리파이 + OLED·RGB LED·버튼 조립, Python GPIO 제어 코드 구현, Firestore 실시간 동기화 (Wi-Fi 연동), 앱 ↔ 라즈베리파이 이벤트 피드백 테스트
4	하드웨어 제작 및 연동																	실사용자 5명 이상 베타 테스트 진 행, 지각 예측 정확도 측정 (오차 ±3분 목표), 준비시간 학습 알고리즘 검증, 날씨·버스 혼잡도 보정값 튜닝
5	통합 테스트 및 고도화																	프로젝트 최종 보고서 작성, 시연 영상 및 발표 자료 제작, 결과물 GitHub 업로드 및 README 작성
6	최종 발표 및 문서화																	

2) 프로젝트 추진 체계

[표 2] 프로젝트 추진체계

성명 / 학번		수행내용	역할
	허재경	- 하드웨어 및 앱 개발 담당 - 라즈베리파이에 OLED 디스플레이, RGB LED(NeoPixel WS2812B), 물리 버튼, 부저, 진동모터를 조립하고 회로도 및 GPIO 핀 맵을 설계함	팀장
	20212999	- Python(RPi.GPIO, rpi_ws281x, Firebase Admin SDK)으로 GPIO 제어 코드 및 알람 로직을 구현함 - Firebase Firestore와 Wi-Fi로 실시간 동기화되도록 구현하며, 버튼 이벤트(알람 끄기·출발 기록)가 앱으로 피드백되는 양방향 통신을 구축함 - 시간 계산 엔진(출발 시각 공식, 날씨 보정, 교통수단별 가중치) 설계를 주도하고 앱·하드웨어 전체 아키텍처의 정합성을 검토함 - 통합 테스트 단계에서 앱·하드웨어 간 연동 오류를 점검하고, 예측 오차 ±3분 이내 달성을 목표로 전체 시스템을 최종 검수함	
	곽우석	- 앱 개발 담당 - Flutter로 메인 화면(실시간 상태 게이지 UI), 캘린더 출결 화면, 설정 화면을 구현함 - Firebase Auth-Firestore와의 데이터 연동 코드를 작성하고, 사용자 시간표·위치·준비시간 데이터가 정상적으로 저장·조회되는지 검증함 - FCM / APNs 푸시 알림 로직을 구현하여 준비 시작 시각에 자동 알림이 발송되도록 개발함 - Kakao Maps API 및 OpenWeather API를 앱에 연동하여 실시간 이동시간과 날씨 보정값이 계산 엔진에 반영되도록 함 - 지각 시 택시 마지노선 자동 계산 기능 및 출발·도착 버튼을 통한 실측 데이터 기록 기능을 구현함	팀원
	20213021		
	김종호	- 데이터·기획 담당 - 프로젝트 초기에 부산 지역 대학생 20명 이상을 대상으로 설문지를 설계하고, SNS를 활용하여 배포한 후 배포일로부터 2주 이내에 수집함 - 수집된 설문 데이터를 통계 처리하여 평균 준비시간·지각 원인(준비 지연·교통 지연·날씨)을 분류하고, 분석 결과를 시간 계산 엔진의 초기 파라미터 설정에 반영함 - 서비스 흐름도, 아키텍처 설계도, 화면 구성 문서 등 기획 산출물을 작성하고 팀 내 공유함 - 베타 테스트 단계에서 실사용자 5명 이상을 모집하여 2주간 실사용 데이터를 수집하고, 예측 오차 및 사용성 피드백을 정리하여 고도화 방향에 반영함 - 최종 보고서 작성 및 발표 자료(시연 영상 포함) 제작을 주도하고, 결과물을 GitHub에 업로드하여 README를 작성함	팀원
	20200372		

III. 결론

1. 프로젝트 결과의 활용방안

본 프로젝트의 결과물은 개인 맞춤형 시간 관리 서비스로서 다양한 분야에 활용될 수 있다. 먼저, 대학생을 주요 대상으로 한 등교 시간 관리 서비스에 적용하여 지각을 예방하고 효율적인 일상 관리를 지원할 수 있다. 이후 서비스 고도화 및 확장을 통해 직장인을 대상으로 한 출근 시간 관리 서비스로 확대 적용이 가능하다. 또한 기존 지도 기반 내비게이션 서비스와 결합하여 단순 이동 시간 안내를 넘어, 사용자 행동 패턴을 반영한 개인 맞춤형 출발 시간 추천 기능으로 확장할 수 있다.

더 나아가 IoT 디바이스와의 연동을 통해 스마트 홈 환경에서 자동 알람 및 생활 패턴 관리 시스템으로 활용 가능하며, 웨어러블 기기와 결합하여 사용자 상태(수면, 활동량 등)를 반영한 고도화된 서비스로 발전시킬 수 있다.

공공 분야에서는 교통 혼잡 완화 및 통학·출근 시간 분산 정책 수립을 위한 데이터 기반 서비스로 활용될 수 있으며, 학교 및 기업에서는 구성원의 출결 관리 및 시간 관리 솔루션으로 적용 가능하다.

2. 프로젝트 결과의 기대성과

기술적 측면	본 프로젝트를 통해 사용자 행동 데이터와 실시간 외부 데이터를 결합한 개인 맞춤형 시간 예측 기술 및 IoT 연동 기반 데이터 수집 기술 확보가 가능하다.
경제적 측면	개인 맞춤형 시간 관리 서비스 수요 증가에 따라 모바일 및 IoT 기반 서비스로 확장하여 수익 창출이 가능하며, 다양한 산업 분야로의 확장을 통한 경제적 부가가치 창출이 기대된다.
사회적 측면	개인의 지각 감소 및 시간 관리 효율 향상을 통해 생산성 증가에 기여하며, 교통 혼잡 완화 및 사회적 비용 절감 효과를 기대할 수 있다.

[참고문헌]

[앱 개발]

- [1] Google LLC. "Flutter Documentation," Internet: <https://docs.flutter.dev>. 2024, [2026년 3월 21일 접속].
- [2] Google LLC. "Cloud Firestore 시작하기," Internet: <https://firebase.google.com/docs/firestore/quickstart>. 2024, [2026년 3월 22일 접속].
- [3] Google LLC. "Firebase Cloud Messaging 개요," Internet: <https://firebase.google.com/docs/cloud-messaging>. 2024, [2026년 3월 22일 접속].

[외부 API]

- [4] Kakao Corp. "Kakao Developers — 지도 API 문서," Internet: <https://developers.kakao.com/docs/latest/ko/kakaomap/common>. 2024, [2026년 3월 24일 접속].
- [5] OpenWeather Ltd. "Current Weather Data API Documentation," Internet: <https://openweathermap.org/current>. 2024, [2026년 3월 24일 접속].

[하드웨어]

- [6] Raspberry Pi Foundation. "Raspberry Pi Documentation," Internet: <https://www.raspberrypi.com/documentation>. 2024, [2026년 3월 26일 접속].
- [7] B. Croston. "RPi.GPIO Python Library — Wiki," Internet: <https://sourceforge.net/p/raspberry-gpio-python/wiki/Home>. Feb. 6, 2022, [2026년 3월 26일 접속].
- [8] PyPI. "rpi-ws281x: NeoPixel LED control library for Raspberry Pi," Internet: <https://pypi.org/project/rpi-ws281x>. 2023, [2026년 3월 26일 접속].

[서비스 기획 참고]

- [9] 공공데이터포털. "기상청 초단기실황조회 서비스," Internet: <https://www.data.go.kr/data/15084084/openapi.do>. 2023, [2026년 3월 22일 접속].
- [10] 공공데이터포털. "국토교통부 대중교통 이용행태 조사," Internet: <https://www.data.go.kr>. 2023, [2026년 3월 22일 접속].